Octopus — comandos e diagnóstico v1.3.0

Revisão documental: **31/10/2026**. Histórico local e catálogo de todas as placas.

O painel HTTP (porta 80) exibe hardware, firmware, memória, rede, relógio, armazenamento e diagnóstico de reinicialização. Também funciona no portal OCTOPUS_CONFIG_PORTAL, usando o endereço do ponto de acesso.

Catálogo de comandos UDP

Envie um comando textual por datagrama à porta configurada, padrão **4210**. Os nomes são normalizados para minúsculas; argumentos preservam a caixa. A Serial mostra logs, mas não há leitor de comandos Serial neste projeto. O limite interpretado é 255 bytes; datagramas maiores são **truncados** pelo receptor atual, portanto não envie comandos acima desse tamanho.

Área	Comandos
Sistema	`help` · `name` · `info` · `status` · `version` · `build` · `reason` · `crash_info` · `alive` · `reboot` · `reset_wifi` · `reset_counters` · `uptime`
Hardware	`temp` · `cpu` · `ram` · `heap` · `flash`
Rede	`net_info` · `mac` · `rssi` · `ip` · `wifi_list`
Relógio	`time` · `date`
Clima	`age`
LED	`led_on` · `led_off` · `led_blink[:ms]`
SD	`sd_status` · `sd_list` · `sd_log` · `sd_clear_log` · `sd_size:/arquivo` · `sd_read:/arquivo` · `sd_write:/arquivo:texto` · `sd_del:/arquivo`

`led_blink` alterna o LED no intervalo informado, padrão 1000 ms, mínimo 50 ms. `sd_list` lista a raiz; `sd_status` informa disponibilidade, não capacidade. `sd_read` e `sd_log` enviam o arquivo por UDP em sequência, entre marcadores de início/fim. Aguarde todas as respostas.

Exemplos de consulta:

```
info
net_info
reason
age
sd_list
sd_read:/udp_log.txt
```

Não há handlers `wifi_add`, `sd_append`, `sd_test`, `sd_mkdir` ou `sd_rmdir` nesta variante. Configure as redes no portal. Escrita/exclusão, reset de configuração, reinício e leitura/apagamento de crash têm efeitos reais; use somente rede confiável.

Prometheus e configuração persistente

`age` retorna a idade em segundos dos dados de Porto Alegre desde o último sucesso no Open-Meteo. Antes da primeira atualização, informa que o clima ainda não foi sincronizado. Após 30 minutos, indica DESATUALIZADOS; falhas posteriores não reiniciam a idade. É uma consulta ao cache, sem acesso à rede.

O painel permanece na porta 80. A aba **Configurações** → **Prometheus** permite ativar o endpoint `/metrics`, escolher uma porta TCP exclusiva (padrão 9120) e informar o IP/hostname do servidor autorizado a coletar. O endereço vazio permite qualquer coletor da rede. A configuração fica nas prefs, é recuperada no boot e pode ser alterada sem reiniciar ou reenviar credenciais Wi-Fi. O Prometheus consulta o Octopus por pull; o dispositivo não envia texto de métricas à porta 9090 do servidor. Portas 80 e 3232 não podem ser usadas pelo endpoint de métricas.

As métricas incluem identificação, disponibilidade, Wi-Fi, CPU, FreeRTOS, heap, PSRAM, flash, GPIOs usados pelo firmware, requisições e diagnóstico SD. Temperatura e umidade de Porto Alegre usam Open-Meteo com os labels da Central; a idade é exportada em `esp_weather_data_age_seconds`. Energia, sensores ambientais locais, relés, ADC e MQTT sem integração são omitidos, como no ESP32C6. Uso de CPU requer duas amostras válidas de runtime stats do FreeRTOS. Consulte o catálogo e o exemplo de scrape em PROMETHEUS.md. Visão geral do firmware e hardware em README.md.

Todas as amostras Prometheus incluem `device="MAC_DO_OCTOPUS"`, inclusive as métricas SD. Reset usa valor 1 e código no label `reason`; IP inclui `interface`; GPIO usa `pin`; logs SD usam caminhos completos no label `file`. Contadores `application_*` são HTTP; comandos UDP usam `esp_udp_*`. `sdcard_log_records_total` e `sdcard_log_errors_total` atendem aos painéis Log Records e Log Error Records, contando gravações e falhas de registro.

`reset_wifi` apaga rede e relógio, mantendo a configuração Prometheus e os contadores de boot/reset/watchdog persistentes. `reset_counters` zera somente as chaves de contadores no namespace `metric_counts`, preservando a configuração Prometheus. `reboot` reinicia o dispositivo. `version` e o painel mostram a versão v1.3.0.

Diagnóstico de falhas

Envie `crash_info` à porta UDP configurada (padrão 4210). O comportamento segue a Central: consulta o core dump ELF em flash, mostra panic, tarefa, PC, backtrace e SHA256 do ELF e arquiva o relatório no SD como `/crash_YYYYMMDD_HHMMSS.txt` (ou `SEM_NTP` com uptime). Só apaga o core dump após reabrir e conferir integralmente o arquivo. Sem SD ou com falha de gravação, preserva o registro interno. Sem suporte a core dump no firmware, informa explicitamente essa condição. A captura requer core dump ELF em flash habilitado e partição coredump. `reason` consulta o motivo da última reinicialização sem apagar registros.

Histórico de comandos e respostas

`/udp_log.txt` registra cada comando UDP aceito pelo receptor (RX) e cada tentativa de resposta (TX), incluindo conteúdo, data/hora local, uptime, IP/porta do cliente, tamanho e resultado do envio. ENVIADO significa que a pilha UDP aceitou o pacote; UDP não confirma a entrega ao cliente. Antes de sincronizar o relógio, use o uptime para ordenar os eventos. Falhas por fila cheia são registradas. O interpretador aceita até 255 bytes por comando; datagramas maiores são truncados ao limite do receptor.

Ao atingir 512 KiB, o arquivo é movido para `/udp_log_old.txt`, substituindo o histórico anterior. A gravação é serializada entre as tarefas. Sem cartão SD, os comandos continuam funcionando sem persistir histórico. O log geral do sistema continua em `/log.txt`.

- `sd_list`: lista os arquivos e relatórios.
- `sd_read:/udp_log.txt`: consulta o histórico atual.
- `sd_read:/udp_log_old.txt`: consulta a rotação anterior.
- `sd_read:/crash_YYYYMMDD_HHMMSS.txt`: consulta um relatório arquivado.

A leitura de arquivos limita-se ao tamanho inicial para não continuar indefinidamente ao registrar as próprias respostas da leitura do log.